Minor Project Synopsis Report

Design and Implementation of a Mini Relational Database Engine

Project Category: Deep-tech

Projexa Team Id-26E2057

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

in

CSE with Specialization AI & ML (Section -B)

to

K.R Mangalam University

By

CHIRAG(2401730183)
VIKRAM(2401730170)
UTKARSH(2401730076)
ANSH(2401730196)
SAURAV(2401730213)

Under the supervision of
MR. VISHWANIL SUMAN

Department of Computer Science and Engineering
School of Engineering and Technology
K.R Mangalam University, Gurugram- 122001, India

January 2026

INDEX

1.	Abstract	Page No.4
2.	Introduction (description of broad topic)	5
3.	Motivation	6
4.	Literature Review	7
5.	Gap Analysis	8
6.	Problem Statement	9-10
7.	Objectives	11
8.	Tools/platform Used	12
9.	Methodology	13-14
10.	References	15

ABSTRACT

Databases form the backbone of modern software systems, yet most computer science students interact with them only through high-level query interfaces, treating database management systems as black-box solutions. The internal mechanisms such as query parsing, indexing, storage management, and execution planning remain largely unexplored at the academic level.

This project focuses on the design and implementation of a Mini Relational Database Engine that demonstrates the internal working of a database system in a simplified yet functional manner. The proposed system supports essential SQL operations such as CREATE, INSERT, and SELECT, along with indexing mechanisms using tree-based data structures.

The database engine is built from scratch using system-level programming techniques, incorporating modules such as a SQL parser, query planner, execution engine, and file-based storage manager. Instead of prioritizing performance optimization, the project emphasizes correctness, clarity, and architectural transparency.

By implementing these core components, the project provides hands-on exposure to low-level database concepts and bridges the gap between theoretical knowledge and real-world database internals.

KEYWORDS: Relational Database, Query Parsing, Indexing, Execution Engine, Storage Management, Deep-Tech Systems

1. INTRODUCTION

Relational Database Management Systems (RDBMS) are fundamental to almost every modern application, from enterprise software to mobile and web-based systems. Popular databases such as MySQL, PostgreSQL, and Oracle provide powerful abstractions that simplify data handling for developers. However, these systems hide their internal complexity, leading many students to use databases without understanding how queries are parsed, optimized, stored, and executed internally.

Understanding the internal architecture of a database engine is crucial for system-level programmers, database architects, and performance engineers. Core components such as lexical analysis, query planning, indexing structures, and disk-based storage systems form the foundation of efficient data processing.

This project introduces a Mini Relational Database Engine that exposes these hidden layers by implementing them explicitly. The system allows users to interact through a command-line interface (CLI) while internally demonstrating how SQL queries are processed step by step. The project serves as an educational tool to demystify database internals and strengthen system design skills.

2. MOTIVATION

The motivation for this project arises from the gap between database usage and database understanding in academic curricula. While students are proficient in writing SQL queries, very few understand how those queries are interpreted and executed internally.

Treating databases as black-box systems limits problem-solving abilities, especially in areas such as performance tuning, query optimization, and system design. With the growing demand for system-level engineers and database developers, it is essential to gain hands-on experience in building foundational software systems.

This project was undertaken to provide practical exposure to:

- How SQL queries are parsed and transformed into executable plans
- How indexing improves data retrieval
- How data is stored persistently on disk
- How execution engines process relational operations

By building a working database engine from scratch, this project enhances conceptual clarity and prepares students for advanced roles in systems and backend engineering.

3. LITERATURE REVIEW

Relational Database System Architecture

Traditional RDBMS architectures are composed of multiple layers including a query parser, optimizer, execution engine, and storage manager. Research literature highlights that modular database design improves maintainability and scalability while enabling optimization at each stage.

Query Parsing and Execution

SQL parsing techniques using tools such as ANTLR and Bison have been widely adopted to convert user queries into abstract syntax trees (AST). Execution engines then interpret these trees to perform relational operations such as selection and projection.

Indexing Techniques

Tree-based indexing structures such as B+ Trees are commonly used due to their efficiency in disk-based storage systems. These structures minimize disk access and provide logarithmic search time.

Educational Database Engines

Several academic projects and lightweight engines have been developed to teach database internals. These systems focus on correctness and clarity rather than performance, aligning closely with the goals of this project.

4. GAP ANALYSIS

Existing database systems prioritize performance, scalability, and fault tolerance, making their internal workings complex and difficult for beginners to understand. While many academic resources explain database concepts theoretically, few provide a complete, hands-on implementation that students can study and modify.

Most existing projects either:

- Focus only on SQL execution without storage internals, or
- Implement storage without a full query pipeline

This project addresses these gaps by integrating **query parsing, indexing, storage management, and execution** into a single coherent system. It provides a balanced implementation suitable for educational and experimental purposes

5. PROBLEM STATEMENT

Modern database systems are powerful but opaque, preventing students from understanding their internal operations. There is a need for a simplified relational database engine that clearly demonstrates how SQL queries are parsed, planned, and executed while managing persistent data storage and indexing.

The challenge is to design and implement a functional database engine that supports core SQL operations and indexing, without focusing on large-scale optimization, but ensuring correctness and architectural clarity

5. OBJECTIVES

- To understand the internal architecture of relational database systems
- To design and implement a SQL query parser
- To build a query planner and execution engine
- To implement file-based data storage mechanisms
- To design and integrate indexing using B+ Tree structures
- To provide a CLI-based interface for user interaction
- To validate system correctness through testing and demonstrations
- To gain a comprehensive understanding of the internal architecture of relational database management systems.
- To meticulously design and implement a robust SQL query parser.
- To construct a highly efficient query planner and execution engine.
- To successfully implement file-based data storage and retrieval mechanisms.

To design and seamlessly integrate indexing capabilities utilizing B+ Tree data structures.To furnish a command-line interface (CLI) for professional user interaction.

To rigorously validate the system's correctness through comprehensive testing and formal demonstrations.

6. Tools/Technologies Used

- Programming Language

  Java for implementing the core database engine due to their strong support for system-level programming and memory management.

- SQL Parsing

  Flex/Bison or ANTLR to tokenize and parse SQL queries into structured representations.

- Storage Management

  File-based binary storage for persistent data handling.

- Indexing

  B+ Tree data structure for efficient searching and indexing.

- Query Processing

  Custom Query Planner and Execution Engine for handling relational operations.

- Interface

  Command Line Interface (CLI) for executing SQL commands.

- Build & Deployment

  CMake / Maven for project build management

  Docker for containerized deployment

- Testing

  Google Test / JUnit for unit testing and validation.

7.METHODOLOGY

The project follows a modular and layered architecture:

1. Query Input
   User inputs SQL queries through a CLI interface.

2. Parsing Phase
   The SQL parser tokenizes and converts queries into an Abstract Syntax Tree (AST).

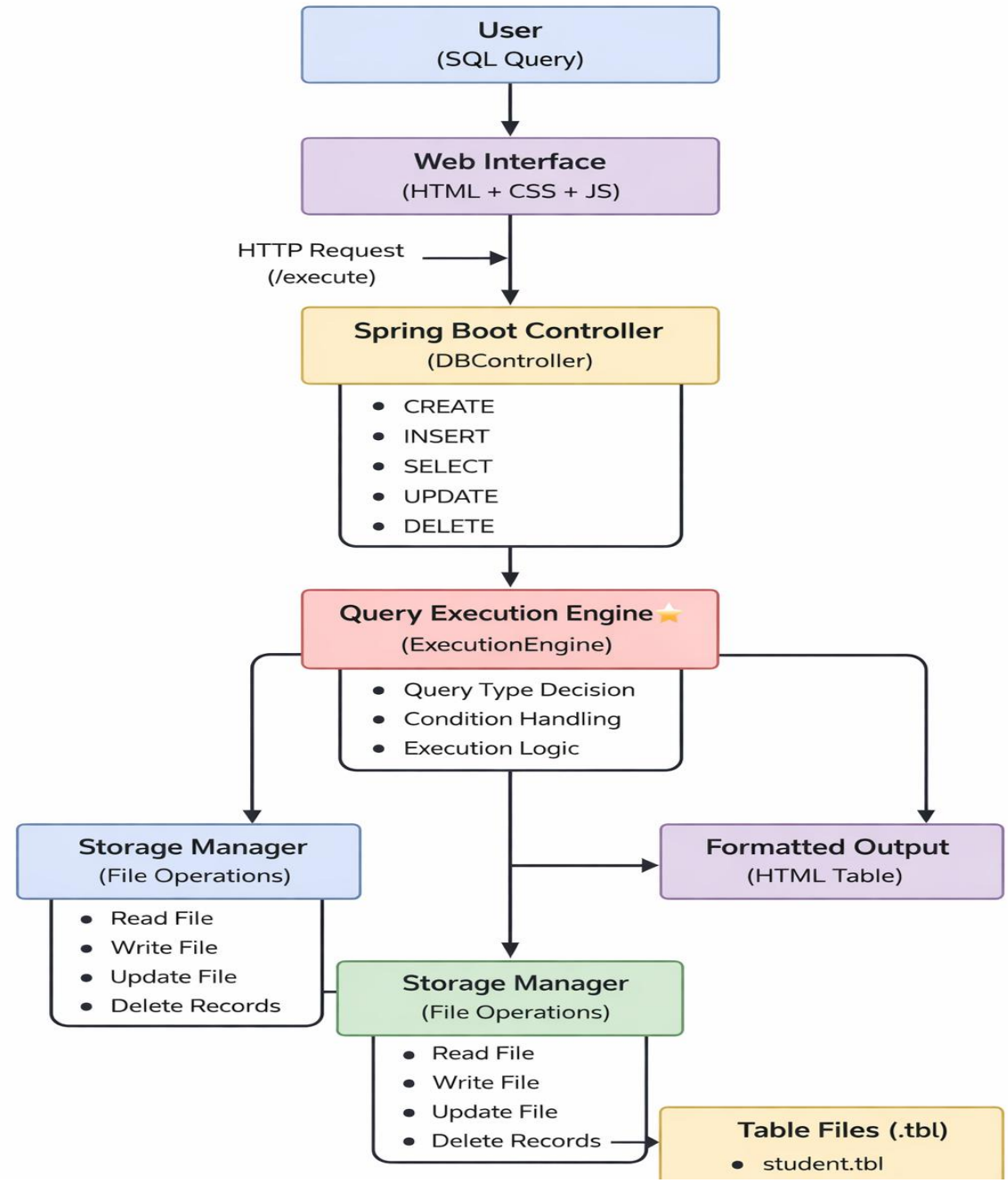
3. Query Planning
   The query planner analyzes the AST and generates an execution plan.

4. Execution Engine
   The execution engine processes relational operations such as selection and insertion.

5. Storage Manager
   Data is stored and retrieved from binary files on disk.

6. Index Manager
   Indexes are maintained using B+ Trees to speed up data access.

7. Result Output
   Query results are displayed to the user, ensuring correctness and consistency.


User
(SQL Query)
Web Interface
(HTML + CSS + JS)
HTTP Request
(/execute)
Spring Boot Controller
(DBController)
CREATE
INSERT
SELECT
UPDATE
DELETE
Query Execution Engine
(ExecutionEngine)
Query Type Decision
Condition Handling
Execution Logic
Storage Manager
(File Operations)
Read File
Write File
Update File
Delete Records
Formatted Output
(HTML Table)
Storage Manager
(File Operations)
Read File
Write File
Update File
Delete Records
Table Files (.tbl)
student.tbl

REFERENCES

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*.
2. Ramakrishnan, R., & Gehrke, J. *Database Management Systems*.
3. Garcia-Molina, H., Ullman, J. D., & Widom, J. *Database Systems: The Complete Book*.
4. ANTLR Documentation – SQL Parsing Techniques
5. Research papers on B+ Tree indexing and storage engines
6. MR. VISHWANIL SUMAN